FlakeRepro: Automated and Efficient Reproduction of Concurrency-Related Flaky Tests

Tanakorn Leesatapornwongsa
Microsoft Research
Washington, USA
taleesat@microsoft.com

Xiang Ren
University of Toronto
Toronto, Canada
jenny.ren@mail.utoronto.ca

Suman Nath
Microsoft Research
Washington, USA
sumann@microsoft.com

ABSTRACT

Flaky tests, which can non-deterministically pass or fail on the same code, impose significant burden on developers by providing misleading signals during regression testing. Microsoft developers consider flaky tests as one of the top two reasons for slowing down software development. In order to debug the root-cause of a flaky behavior, a developer often needs to first reliably reproduce a failed execution. Unfortunately, this is non-trivial. For example, most of the flakiness in unit tests are caused by concurrency, and reproducing their failures requires specific thread interleaving. To address this challenge, we introduce FlakeRepro that helps developers reproduce a failed execution of a flaky test caused by concurrency. FlakeRepro combines static and dynamic analysis to quickly identify an interleaving that makes a flaky test fail with the same original error message. FlakeRepro is efficient: it can reproduce a failed execution after exploring few tens of interleavings. FlakeRepro integrates well with existing systems: it automatically instruments test binaries that can run on existing and unmodified test pipelines.

We have implemented FlakeRepro for .NET and used it in Microsoft. In an experiment with 22 Microsoft projects, FlakeRepro could reproduce 26 of total 31 concurrent-related flaky tests, after exploring only < 7 interleavings and within 6 minutes on average.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

flaky tests, debugging and fault localization, program analysis, software testing

ACM Reference Format:
Tanakorn Leesatapornwongsa, Xiang Ren, and Suman Nath. 2022. FlakeRepro: Automated and Efficient Reproduction of Concurrency-Related Flaky Tests. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22), November 14–18, 2022, Singapore, Singapore.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3540250.3558956

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9413-0/22/11…$15.00
https://doi.org/10.1145/3540250.3558956

1 INTRODUCTION

Flaky tests, which can non-deterministically pass or fail on the same code, are prevalent in both industry and open-source community [7, 8, 12, 22, 29]. They impose a significant burden on developers, especially while using the continuous integration (CI) model. For example, flaky tests can seriously undermine regression testing by providing misleading signals about the quality of a developer's recent changes. As reported in [8], Microsoft developers consider flaky tests as one of the top two reasons for slowing down software development. Consequently, debugging and fixing existing flaky tests is one of their top three actions in fighting the flaky tests.

Identifying the root cause of a test's flakiness is challenging. Often, the test's failure message and stack trace indicate only the symptom, without any obvious hints pointing to the underlying root cause (*e.g.*, the example in Section 2.3). In order to debug such flakiness, a developer often needs to reproduce a failed execution so that she can enable extensive logging, or attach a debugger to step through runtime states and behavior. Unfortunately, such reproduction is non-trivial since most flaky tests fail rarely or only in a specific test environment. Moreover, enabling logging or attaching a debugger may make certain flakiness disappear, making the debugging task even harder [8]. No practical tools or techniques exist today that can reliably reproduce a failed execution outside the test environment, with extensive logging enabled, and/or with a debugger attached. As a result, frustrated developers often suppress flaky tests, which is a dangerous practice since flakiness in product code can eventually lead to production failure [33].

In this paper, we present FlakeRepro that addresses this challenge. We develop our solution in the context of flaky unit tests executed in Microsoft's CI pipeline called CloudBuild[4]. In this setting, concurrency is the dominant cause of flakiness (86% in a sample reported in [9]; more details in Section 2). Concurrency is also a major root cause of flakiness in open source software, affecting 65% of the flaky tests as reported by [12]. We therefore focus on concurrency-related flaky tests; more precisely, on tests that intermittently fail due to buggy interleavings of concurrent operations. Given such a flaky test and its failure symptom (*e.g.*, error message on assertion failure), FlakeRepro deterministically reproduces a failed execution that generates the same symptom. Since the reproduction is deterministic, a developer can enable extensive logging or attach a debugger to step through the failed execution while reproducing the failure.

Building such a tool that is practical and can be used at scale (*e.g.*, in CloudBuild that runs hundreds of millions of unit tests per day), several challenges must be addressed. First, the tool must explore the space of thread interleaving intelligently, as the number of interleavings grows exponentially with the number of threads

and the number of steps executed by each thread. Existing concurrency bug finding tools such as CHESS[28] and ConSeq[40] use intelligent techniques to prune the space, however, they are not practical at the scale of CloudBuild for two key reasons. First, they rely on expensive optimizations such as *happens-before tracking* that require instrumenting all synchronization primitives such as thread creation and join; recent work has shown that this is not practical for modern languages such as C# that have too many such primitives[16, 17]. Second, despite such optimizations, bug finding tools still need to explore a large number (many thousands in CHESS) of interleavings per test, which can be prohibitively expensive at the scale of CloudBuild. So we must find a solution that does not rely on such expensive optimizations and can expose bugs after exploring a small number of interleavings.

Second, for the tool to be widely used, we must be able to use it with the application under test as is. We cannot expect testers to change their code or to annotate synchronization primitives or to run their tests under a specialized runtime or VM. The tool should also integrate easily with existing test systems, without requiring any changes to the test harness. Finally, it should be easy to use, within the test system or on a developer's local machine.

We present FlakeRepro that addresses the aforementioned challenges. To address the first challenge, we leverage the fact that unlike existing bug finding tools that aim to find any and all possible concurrency bugs, our goal is to find the bug that can reproduce a given failure symptom (*e.g.*, a failure message). Thus we can use intelligent static and dynamic analysis to explore only the interleavings that are likely to reproduce the target symptom. Our static analysis identifies a small number of shared memory accesses whose ordering may affect the failure. Next, our dynamic analysis explores interleavings of only those accesses (instead of interleavings of all instructions). The exploration space of interleavings is further dynamically pruned by novel heuristics that we develop based on the patterns of flaky tests we observe. As a result, FlakeRepro can effectively reproduce a failed execution after exploring a small number of interleavings (in the order of tens).

To address the second challenge, we leverage automatic instrumentation of test and application binaries. FlakeRepro first statically analyzes the binaries to identify a small set of static locations where delays can be injected to generate different thread interleavings. FlakeRepro then automatically instruments the binaries to intercept execution of those static locations and to inject delays. The process does not require source code modification or any inputs from developers, making FlakeRepro a push-button tool.

FlakeRepro works in two phases. In the *exploration phase*, the instrumented test is executed multiple times, with each execution injecting delays at different locations and thereby exploring different thread interleavings. When an execution fails with the target error message, the phase ends and outputs the interleaving of that failed execution. In the *reproduction phase*, FlakeRepro deterministically replays a buggy interleaving produced by the exploration phase. FlakeRepro's instrumentation updates test binaries' symbol files as well so that reproduction works even with a debugger attached. Since FlakeRepro only needs to instrument test/application binaries that can run on unmodified test runtime, it can easily integrate with an existing test pipeline and can run on a developer's local machine as well.

We have implemented FlakeRepro for .NET and we report the result of applying it to 22 projects within Microsoft. The projects contain 31 concurrent flaky tests. FlakeRepro could deterministically reproduce 26 of the 31 flaky tests. The reproduction was quick: half the tests required exploring only 1 or 2 interleavings, and on average, a test took < 6 interleavings to explore.

2 BACKGROUND AND SCOPE

2.1 Flaky Tests in Microsoft

Prior work [8, 9] offers a good description of flaky tests in Microsoft. In summary, flaky tests are prevalent. For instance, they account for 27% of the build failures in a collection of five projects reported in [8]. CloudBuild [4], Microsoft's CI pipeline, helps a developer automatically identify flaky tests with an opt-in feature called Flakes. Flakes identifies flaky tests by retrying a test when it fails. If a failed test succeeds on retry, it is labeled as flaky and a bug report is automatically created for the developer to investigate the flakiness. Unfortunately, many flaky tests reported by Flakes are not fixed, mainly because developers cannot reproduce the failures locally. In such cases, developers usually suppress the flaky tests to avoid intermittent failures of future builds. However, this is a dangerous practice since the underlying bugs can be in product code and can manifest at large-scale production. This motivates us to work on practical tools that can help developers reproduce flakiness for debugging.

The key underlying reason for flakiness of unit tests in CloudBuild is concurrency (86% in a sample reported in [9]). As reported in [20], concurrency is a dominant root cause of flakiness in open source application as well (65%). We therefore prioritize our focus on reproducing concurrency-related flakiness.

2.2 Scope of the Paper

Bug types. In this paper, we focus on a specific but important class of concurrency bugs that cause test flakiness. In particular, we focus on bugs that are triggered by a specific interleaving of two threads accessing a shared memory location. The bugs include data races (on shared memory), order violation, atomicity violation, etc. We do not consider other types of concurrency such as deadlocks or races on a non-memory object such as a file or database. Two common patterns of concurrency-related flaky tests that FlakeRepro can reproduce are as follows.

Sleep/timeout. In this pattern, a test starts an asynchronous operation and instead of using proper synchronization to ensure its completion, it waits for a specific period and assumes that the asynchronous operation has completed within that period. The test fails when the operation does not complete within the fixed wait period. The pattern is similar to the Async Wait pattern reported in [9]. Scenarios involving this pattern can be complex in practice. A test may invoke multiple asynchronous methods and may have multiple sleeps/timeouts. Moreover, asynchronous method invocations and sleeps/timeouts may be deep inside product or library code, making it non-trivial for the developer to reproduce.

Data race. Another common pattern is *data race* caused by multiple threads accessing the same instance of object without proper synchronization. A data race can non-deterministically lead to data

corruption and test failure. Reproducing them can be difficult because there can be many such thread-unsafe operations and they may or may not race depending on how/whether they are synchronized and how the underlying scheduler schedules them.

Failure reproduction. Our goal is to reproduce a test failure with respect to a given error message. We assume that the error message contains detailed information on the error site (*i.e.*, which class/method/instruction failed) and a stack trace collected during the error, and we aim to reproduce an execution that results in the same error message along with the same stack trace.

For example, when the flaky test in Listing 1 (discussed in Section 2.3) fails with an assertion failure, it produces an error message as below (and our goal is to reproduce the same error message):

```
Failed MarkUnavailableTest
Error: Assert.AreEqual failed. Expected:uri2. Actual:uri1.
StackTrace: MarkUnavailableTest() in EndpointTests.cs:line 10
```

Our definition of reproduction is similar to that of Pensive [42], where the goal is to reproduce the externally visible symptom of a failed execution, rather than to exactly replay all aspects of the failed execution, as is aimed by record-replay systems [2].

Implementation. We implement FlakeRepro for .NET due to its popularity within Microsoft, even though our core algorithms can be implemented in other languages as well.

2.3 A Case Study

We present a case study to showcase how FlakeRepro helped in diagnosing and fixing a real-world flaky test in Microsoft. A simplified version of the test is shown in Listing 1. It creates an instance of `EndPointManager` (`m`) and initializes it with two endpoint URIs. Then it asserts that `uri1` is the first available endpoint in `m`. After that, the test marks `uri1` as unavailable and refreshes `m` to garbage collect the unavailable endpoint `uri1`. Finally, it checks if `uri2` is the first available endpoint in `m` (since `uri1` has been marked unavailable and then removed).

Microsoft's test system reported that the test intermittently failed with the error message: `Assert.AreEqual failed. Expected:uri2. Actual:uri1.` with a stack trace including line 10. This tells the developer that the second assertion fails, but does not give any specific root cause. In fact, there are multiple possible non-deterministic factors to cause the failure; *e.g.*, the `Init()` method could fail to add `uri2`, `MarkUnavailable()` could fail to mark `uri1` as unavailable, `Refresh()` could fail to remove `uri1` or mistakenly remove `uri2`, etc. The developer first tried to attach a debugger to step through the test to pinpoint a plausible root cause; but unfortunately, the test always passed when the debugger was attached.

Using FlakeRepro. The frustrated developer then used FlakeRepro, by providing the test binaries and the error message to it. After the exploration phase, FlakeRepro reported that the error message could be reproduced with an interleaving where the `Refresh()` method called inside the `Init()` method (in line 4) is completed *after* line 8 is invoked. This output alone was useful to the developer: it was evident that there was another `Refresh()` that non-deterministically interfered with the `Refresh()` in line 8. Next, the developer invoked FlakeRepro with this output to *deterministically*

```
1  [TestMethod]
2  public void MarkUnavailableTest() {
3    var m = new EndPointManager();
4    m.Init(new List<Uri> {uri1, uri2});
5    var ep = m.GetEndPoint(0);
6    Assert.AreEqual(uri1, m.GetEndpoint(0));
7    m.MarkUnavailable(uri1); // uri1 marked unavailable
8    m.Refresh(); // remove unavailable endpoint uri1
9    ep = m.GetEndPoint(0);
10   Assert.AreEqual(uri2, ep);
11 }
12
13 class EndPointManager {
14   Cache cache;
15   bool isRefreshing;
16   object refreshLock;
17
18   public Uri GetEndPoint(int idx)=>cache.GetData(idx);
19
20   public void Refresh() {
21     if (isRefreshing)
22       return;
23     lock (refreshLock) {
24       if (isRefreshing)
25         return;
26       isRefreshing = true;
27     }
28     cache.Refresh();
29     lock (refreshLock) {
30       isRefreshing = false;
31     }
32   }
33 }
```

Listing 1: A flaky test in Microsoft

reproduce the error. In addition, the developer could attach a debugger to step through the test execution while it was deterministically failing to investigate how the two `Refresh()` calls were interfering.

The root cause. The developer found that the `Init()` method creates a background garbage collection thread that periodically invokes `Refresh()` to remove URIs marked as unavailable. A race condition is avoided by ensuring that only one `Refresh()` runs at a time: if an invocation of `Refresh()` sees that another refresh is ongoing, it immediately returns (line 22) and let the current or the next refresh delete the unavailable URIs. Thus, whether the `Refresh()` in line 8 actually removes `uri1` depends on whether there is an ongoing refresh or not. This is precisely the reason why the assertion on line 10 non-deterministically fails: if the background `Refresh()` started by `Init()` is completed before (or has not started before) line 8, the `Refresh()` on line 8 succeeds in removing `uri1` and the assertion on line 10 succeeds; otherwise the assertion fails.

The fix. After FlakeRepro helped diagnose the root cause, the developer also came up with the fix to avoid the problematic interleaving produced by FlakeRepro's exploration phase. The fix was a *blocking* version of `Refresh()` that would first wait for any ongoing refresh to finish and then would proceed with its original functionality.

3 FLAKEREPRO

This section describes the design of FlakeRepro. Given a test binary and a flaky error message, FlakeRepro works in two phases. In the *exploration phase*, it identifies a buggy thread interleaving

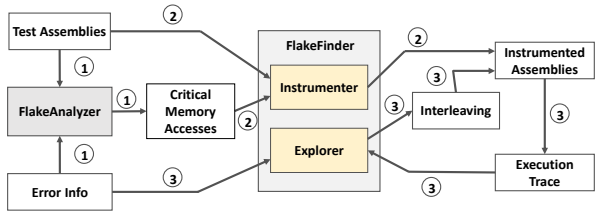


Figure 1: The architecture of FlakeRepro and the workflow of how it works.

that causes the test to fail with the given error message. In the *reproduction phase*, FlakeRepro enforces that particular interleaving so that a developer can deterministically reproduce the error.

Exploration phase. To identify a buggy interleaving, FlakeRepro must explore the space of thread interleaving intelligently, as the number of interleavings grows exponentially with the number of threads and the number of steps executed by each thread. A key novelty of FlakeRepro is to significantly reduce this space via the two mechanisms. First, we observe that many concurrency-related test assertion failures involve shared memory accessed by multiple threads, and some interleavings of the accesses may make the assertion to fail. Therefore, instead of exploring all possible thread interleavings, FlakeRepro only explores interleavings of shared memory accesses that influence the target error message through data- and control-flows. As our experiments show, this under-approximation of the state space works reasonably well in practice, while reducing the state space significantly. Second, while exploring the state-space, FlakeRepro observes runtime behavior of the test and uses it as feedback to further reduce the state space. We propose two such heuristics based on common patterns of flaky tests we see. Combination of these two strategies make FlakeRepro efficient—often requiring only few tens of interleavings to explore to find the buggy interleaving.

Figure 1 shows FlakeRepro's architecture. It consists of two key components: *FlakeAnalyzer* and *FlakeFinder*. The following workflow describes how they work during the exploration phase:
(1) *FlakeAnalyzer* uses static data- and control-flow analysis of the test's binaries to identify *critical memory accesses* that affects the target error message. (Shown as (1) in Figure 1.) FlakeRepro limits its exploration only to the interleavings of the critical accesses.
(2) The *Instrumenter* component in *FlakeFinder* instruments test binaries to intercept on and control interleaving of their critical accesses (shown as (2)).
(3) The *Explorer* component in FlakeFinder systematically explores the interleavings of the critical accesses by executing the instrumented test binaries (shown as (3)). The exploration space is pruned by feedback of executions. When an interleaving leads to the target error message, FlakeFinder stops and produces the interleaving that a developer can later use to reproduce the failure. It is possible that no interleavings FlakeRepro has explored produce the error message, in which case FlakeAnalyzer declares the error to be not reproducible by it.

Reproduction phase. FlakeRepro's exploration phase is relatively simple. Given a buggy interleaving produced by the exploration phase, FlakeRepro intercepts on all instrumented shared memory accesses and ensures that they are executed in the same order as the given interleaving. For example, if access A happens before access B in the buggy interleaving, FlakeRepro intercepts on B and waits until A finishes to start B.

In the rest of the section, we describe the details of FlakeAnalyzer and FlakeFinder.

3.1 FlakeAnalyzer

The goal of FlakeAnalyzer is to statically analyze test binaries to identify a small set of *critical accesses* whose interleavings will be dynamically explored by FlakeFinder. A critical access is a shared memory access that can potentially affect the test failure through data- and control-flow dependencies. FlakeAnalyzer's analysis is (1) automated: it does not require source code or developer input on application semantics, (2) scalable to complex production code, and (3) able to quickly narrow down to a small set of critical accesses.

The first step in identifying critical accesses is to find all shared memory accesses (which is a super set of critical accesses). Detecting shared-memory accesses requires identifying all instructions which may reference the same memory location. In general, this requires expensive pointer analysis which works poorly on real-world program due to poor scalability and low precision.

The FlakeAnalyzer automates shared-memory access detection in a scalable manner by exploiting the design of modern high-level programming languages that they do not expose raw pointers (*e.g.*, C#, Java, and Python)[1]. This means that threads can access shared memory via accessing the same field of the same object instance only. For example, in Listing 1, all accesses to the fields of `EndpointManager`—`cache`, `isRefreshing`, and `refreshLock`—are considered as shared-memory accesses (*e.g.*, line 18, 21, and 23).

The FlakeAnalyzer identifies accesses of the same field using scalable *fuzzy search* [42] — it carries out a global search for the field accesses in the binaries, *i.e.*, it finds all instructions that access fields of the same type; it does not analyze whether the object references point to the same instance. Thus, if two threads access the same field, but different instances of object, the fuzzy search still identifies those accesses as shared-memory accesses. We find this design is highly effective in practice — FlakeAnalyzer is able to narrow down shared accesses to just a small set of field accesses. Subsequently, we deploy the FlakeFinder to further filter out non-shared-memory accesses through dynamic analysis.

Identifying critical accesses. FlakeAnalyzer further bridges the failed instruction to *critical accesses*. The failed instruction can be any instruction that throws an exception, but for simplicity, we will use a test which fails from an unmet assertion as an example. Starting from the failed assertion, FlakeAnalyzer carries out *selective backward static slicing* to identify dependencies on shared-memory accesses [36]. The process uses the following steps. (1) As the first step, FlakeAnalyzer performs data-flow-only slicing from the values used in the failed assertion by using *reaching definition* analysis [36]. This is performed recursively in a tree traversal manner on the data-flow dependency graph because a value can have

[1]With the exception of the `unsafe` keyword in C#, however, we find minimal usage, if not zero, of exposed pointers in Microsoft.

```
1  input: test, symptom
2  identify_critical_accesses:
3      failed_op = get_failed_op(test, symptom)
4      queue = fifo_queue()
5      for variable in get_used_vars(failed_op):
6          queue.push(failed_op)
7      while queue.is_not_empty():
8          variable = queue.dequeue()
9          if is_field_access(variable):
10             critical_access_list.add(variable)
11             defs = get_control_flow_predes(variable)
12         else if is_local_variable(variable):
13             defs = get_reaching_definitions(variable)
14         else if is_returned_value(variable):
15             defs = get_return_in_callees(variable)
16         else if is_parameter(variable):
17             defs = get_invokes_in_callers(variable)
18         else if is_load_from_addr(variable):
19             defs = get_stores_to_addr(variable)
20         else if is_constant(variable):
21             defs = get_control_flow_predes(variable)
22         for def in defs:
23             for new_var in get_used_vars(def):
24                 queue.push(new_var)
25     return critical_access_list
```

Listing 2: Pseudocode showing Analyzer's algorithm

multiple definitions. FlakeAnalyzer stops analyzing a data-flow dependency after encountering a critical access (in the form of a field access instruction) or a constant definition (which has no further data-flow dependencies). Afterwards, (2) FlakeAnalyzer performs control flow slicing for any field access or constant definition in the current method. FlakeAnalyzer further repeats step (1) and (2) on the condition variables used by branches in the control flow slice.

The algorithm for identifying critical accesses is shown in Listing 2 in pseudocode. We explain algorithm in detail with the example listed in Listing 1. For concreteness, even though FlakeAnalyzer works with the binaries, we will use C# code as an example.

(1) *Data-flow Slicing:* FlakeAnalyzer starts by analyzing variables used by the failed assertion; It first searches for definitions of these variables that can reach the assertion in the *current* method (reaching definition analysis). In Listing 1, the test fails because of `Assert.AreEqual()` at line 10, hence FlakeAnalyzer starts its analysis from the `AreEqual()` method. FlakeAnalyzer analyzes the data-flow dependencies of `uri2` and `ep`. It starts analyzing `uri2` first. For simplicity, we assume that `uri2` is a constant definition (in the real code, `uri2` is an object constructed from a constant string), and since it is constant, it has no more data-flow dependencies — it also does not have any control flow dependencies (proceeding branches), so the analysis for `uri2` ends.

Now FlakeAnalyzer backtracks to analyze `ep`, which is defined by the return value of `m.GetEndPoint(0)` (line 9). FlakeAnalyzer further analyzes the data-flow dependencies in the callee method. It does so by first locating all the return instructions in the callee method and *recursively* performs reaching definition analysis on all statements. However, in this case, there is only one return statement in `GetEndPoint()` (line 18). FlakeAnalyzer sees that the return value is `cache.GetData(index)`, which the value depends on two variables, `cache` and `index`. `cache` is a field member of `EndPointManager`, then the fuzzy search declares it as a *critical access*, FlakeAnalyzer then does control-flow analysis of `cache.GetData(index)`, but it has no control-flow dependencies.

Next, FlakeAnalyzer backtracks to analyze `index`, which is the parameter of the method. To analyze a parameter variable, FlakeAnalyzer keeps the call-stack context as it performs interprocedural data-flow analysis. Specifically, when FlakeAnalyzer traverses down to `GetEndPoint()` to analyze return value, it pushes the callee, `GetEndPoint()`, onto the stack, and pops when finishes analyzing this method. FlakeAnalyzer then can track the definition of `index`, and repeats this data-flow analysis at that location (line 9), which is the constant value 0, so it stops here.

Although not illustrated in this example, another way variables can be passed across methods in .NET applications is through pass-by-reference. In this case, the address of the variable is passed into the callee method, allowing the callee to modify the variable value by storing to the address; the variable is then accessed in the callee method by loading from the address. When FlakeAnalyzer encounters a variable loaded from an address, it knows the variable has been passed-by-reference into a callee, and searches for the callee invocation site. Subsequently, FlakeAnalyzer moves into the callee method to look for instructions that store to the address, repeating step (1) on the store values.

(2) *Control flow Slicing:* FlakeAnalyzer invokes control-flow slicing when it reaches a critical access or a constant definition. Control-flow slicing is performed in the current method by traversing the control flow dependency graph to find all predecessors of the definition or branches that may affect whether the definition gets executed. For example, while FlakeAnalyzer is looking into `EndPointManager.Refresh()`, and analyzing the return statement at line 22, it carries out control-flow analysis and sees that this branch relies on the condition predicate `isRefreshing == true` at line 21. FlakeAnalyzer then repeats step (1) of the data-flow analysis to further analyze all the variables in the condition, after which it will declare that `isRefreshing` is another critical access because it is a field of `EndPointManager` class.

Note that while identifying critical accesses, FlakeAnalyzer does not do array indexing analysis, and treats an array as another object. The granularity of memory access when working with an array is the array reference, not individual elements.

3.2 FlakeFinder

After FlakeAnalyzer identifies a set of critical accesses that may affect the test outcome, FlakeFinder dynamically explores the space of interleavings of critical accesses to find a buggy interleaving that causes the test failure.

Instrumenter. For FlakeFinder to explore different interleavings of critical accesses, it needs to control the timing of their executions. One way to achieve this is to modify the underlying scheduler of test runtime; however, that would require the tests to be run in specialized runtime, complicating FlakeRepro's production deployment and local usage. FlakeRepro therefore relies on instrumenting test binaries so that critical accesses can be intercepted and interleaved in a desirable way during runtime. This is achieved by the *Instrumenter* sub-component of FlakeFinder: given test binaries and a list of critical accesses, it produces an instrumented set of binaries with each critical accesses instrumented (ⓐ in Figure 1).

```
1  public void Refresh() {
2      FlakeRuntime.UpdateExecIndex("Refresh", loc1);
3      FlakeRuntime.Before("Refresh", loc1);
4      bool tmp = isRefreshing;
5      FlakeRuntime.After("Refresh", loc1);
6
7      if (tmp) {
8          FlakeRuntime.UpdateExecIndex("Refresh", loc2);
9          return;
10     } // the rest is omitted due to space limit
11     ...
12 }
```

Listing 3: The instrumented code of `Refresh()` from Listing 1 for controlling the timing of critical accesses

Instrumenter injects instructions before and after the critical access instructions. An example of injected code of `Refresh()` from Listing 1 is shown in Listing 3. `FlakeRuntime` is FlakeRepro's runtime library that is installed in the test directory during instrumentation. `FlakeRuntime.Before()` acts as a synchronization guard to delay the critical accesses of `isRefreshing`, if needed. It also records a stack trace when a thread is about to execute the following critical-access instruction. This stack trace is not required to reproduce the error, but is a useful information for a developer while debugging the flaky test. For `FlakeRuntime.After()`, it acts as `Notify()`, and records the execution trace that will be used as a feedback during the exploration phase.

Note that while exploring different interleavings of critical accesses, FlakeRepro must uniquely identify dynamic instances of critical accesses. This is required for FlakeRepro to avoid executing duplicate interleavings and to replay a buggy interleaving discovered by it. FlakeRepro uses *execution indexing* [37] to uniquely identify such instances. During the instrumentation, the Instrumenter injects code to compute execution index of each critical access. A call to `FlakeRuntime.UpdateExecIndex()` needs to be injected before all critical accesses, before every method invocation, and at the beginning of all basic blocks, as shown in Listing 3. More details of how FlakeRepro uses the execution index will be described later.

Explorer. The *Explorer* sub-component of FlakeFinder is responsible for exploring different interleavings of the critical accesses. Its main task is searching for an order of critical accesses that fails the test with the given error message. The Explorer chooses an order it wants to check, configures the instrumented binaries to enforce that order, starts the test runner, and checks the result to see if it fails with the target error message. The process is repeated until it finds a buggy ordering that generates the error message. For each round, it also gathers the execution trace as a feedback to make a decision for the next round. The workflow of Explorer is a loop labeled with (3) in Figure 1.

Note that the critical accesses identified by the FlakeAnalyzer are static locations. They alone are not sufficient for Explorer to select an interleaving to explore. First, one static critical access location can be executed multiple times (*e.g.*, a critical access inside a loop) and the Explorer must consider the ordering between *dynamic instances* of a critical access. Second, some static critical accesses may not be executed by a test, or may be executed only sequentially and Explorer do not need to consider them while choosing a target interleaving. To identify the concurrent critical access instances, the Explorer first executes the instrumented test without any order enforcing to gather an execution trace. Explorer then analyze the trace to identify all concurrent instances, and then start searching for the buggy interleaving of those access instances.

Consistent IDs of critical access instances. To refer to the same critical access instance across multiple runs, Explorer needs to assign to each instance a *unique* and *consistent* ID, called *access ID*. The ID should uniquely identify the access instance within the same run and the same logical access instance across runs should consistently have the same ID. An ID based on the access' static location alone will not be unique across instances while a random ID for each access instance may not be consistent across runs. To address this, we adopt the idea of execution indexing [37] that gives a unique and consistent ID to each executed instruction across runs, and Instrumenter will inject a call to compute the execution index before every critical access (*e.g.*, `UpdateExecIndex()` at line 2 computes the access ID of the access at line 4 in Listing 3).

Explorer computes access ID as a combination of the consistent ID of the current thread, and the *execution context* within the thread for that instance. The execution context consists of the following: (1) The static location (instruction address). (2) The basic block ID which is assigned by us. (3) The call stack of the access instance within the thread. (4) If the access location is nested within any loops, the iteration of each loop the access falls will be included.

The consistent thread ID needs to remain the same across runs. Clearly, thread ID assigned by the underlying runtime may not be consistent since the same logical thread might have different IDs across different runs (e.g., in .NET). Conceptually, an execution index computed at a thread's spawn point can be used as its consistent ID. A thread's spawn point can be dynamically detected by instrumenting the underlying runtime's thread manager and by utilizing its threading model and synchronization primitives, and/or by developers annotating application code. However, FlakeRepro is designed to work as a push-button tool—on unmodified runtime and without developer annotation/input. Therefore, Explorer does not know where all the thread spawn points are in the code.

To solve this problem, we leverage a special type of thread-local storage that is available in many modern programming languages such as Java and .NET and that is automatically passed from a parent thread to its child thread. We describe an implementation of the idea in details in Section 4.

Exploration algorithm. In theory, for a concurrency bug to crash a system, it is possible that the bug requires an exact order of all critical accesses instances. For example, consider a test that has 5 threads and each thread accesses the same object. There will be 5 access instances during the runtime (AI1-AI5). Theoretically, for the failure to manifest, it might need these instances to run strictly with the order [AI2, AI4, AI1, AI5, AI3]. In this case, we need to explore all permutations which are 5! or 120 trials to expose the failure. Even worse, in reality, there will be more than 5 instances to permute and this will lead to the state space explosion problem.

However, in reality, it is unlikely that the test will require all access instances to be in an exact order. The previous studies on concurrency bugs found that majority of concurrency bugs require

```
1  input: instrumented_test, symptom
2  explore():
3      exec_trace = run_test(instrumented_test)
4      access_inst = extract_instances(exec_trace)
5      while true:
6          order = get_access_inst_pair(access_inst)
7          if order is null:
8              return "Cannot Reproduce"
9          result, trace = run_test_with_order_enforce(
10             instrumented_test, order)
11         if result == symptom:
12             return order
13         else:
14             reprioritize(access_inst, order, trace)
```

Listing 4: Pseudoecode showing Explorer's algorithm

only a pair of instances or critical events to be in a certain order for failure to expose [15, 18] (*e.g.*, if AI2 executed before AI1, the test will fail, no matter which order the other instances are). Based on the statistics, the Explorer chooses to pick a pair of critical access instances and enforce the partial order of those two only, hence, FlakeRepro cannot reproduce a failure caused by a thread interleaving involving three access instances or greater. However, we see that this partial-ordering-based exploration algorithm works well in our evaluation (Section 5). Also, because of this partial-ordering exploration, it is possible that the Explorer will explore one interleaving more than once on different partial orders.

Listing 4 shows the pseudocode of how Explorer works, it chooses a pair of the access instances to try with prioritization strategies (discussed later in this section). Explorer prioritizes some candidates based on previous iterations by `get_access_inst_pair()` at line 6, and after the current iteration, it re-prioritizes the candidates by calling `reprioritize()` at line 14.

Intelligent exploration with feedback. As aforementioned, FlakeAnalyzer will not need the semantic of .NET multi-threaded programming model, and this comes with the cost that Explorer cannot determine happens-before relationship of each access instances [13, 21], so Explorer might try to enforce the access instance AI2 before AI1, but AI1 has happens-before relationship with AI2, and Explorer will fail to do that.

To minimize the number of waste attempts due to impossible ordering, Explorer constructs happens-before relationship graph while exploring dynamically. When it sees that it cannot enforce AI2 before AI1 despite delaying AI1, it puts a happens-before edge from AI1 to AI2. Explorer also detects happens-before relationships by examining the resultant delay of each critical access instance. Say Explorer postpones the execution of AI1 by D ms. If AI1 has a happens-before relationship with another access instance AI2, then AI2 must have a delay approximately D ms as well. This mechanism has a similar concept introduced by TSVD [17]. As a result, Explorer collects a timestamp at each instance, and infer the happens-before relationship. Explorer uses this information to avoid redundant future exploration of impossible schedules.

Exploration heuristics. Indeed, we observe that most concurrency-related flaky tests occur through the pattern of Async Wait from Luo et al.'s study [20]. Specifically, when a test fails because of an unmet assertion, we see the *assertion* reads a critical shared variable, and the absence of a proper prior write — caused by a delay in the real-world server environment — causes the assertion to fail non-deterministically. We guide Explorer's exploration strategy according to this pattern by enumerating all critical access instance pairs that the former one is *fixed* to be the instance from the failed assertion, and the latter one is another instance, then we prioritize to explore these orderings first. If after these pairs are all explored, and Explorer cannot still expose the error, it then explores other arbitrary pairs.

Another heuristic that we apply is that if we have tried the order [AI1, AI2], and if AI1 is a critical access in a loop, we will de-prioritize the orders [AIx, AI2] where AIx instance is in the same loop but different iteration, because we observe that if AI1 does not affect the test outcome, the other instances will be likely the same.

4 IMPLEMENTATION

We have implemented FlakeRepro for .NET. Both FlakeAnalyzer and FlakeFinder are written in C#. FlakeAnalyzer uses Analysis.NET [43] for static analysis. Analysis.NET is a library for analyzing .NET binaries that provides common static analysis functions such as data-flow analysis, control-flow analysis, and call-graph analysis. FlakeFinder uses Mono.Cecil [5] to instrument .NET binaries.

Consistent thread ID. We now describe how our implementation computes a consistent thread ID. As discussed in Section 3.2, an execution index computed during runtime at a thread's spawn point can be used as a consistent thread ID. However, as per our design goal, we do not instrument .NET runtime or require developer annotations/inputs, and hence FlakeRepro does not know the precise spawn point of a thread. We address this challenge by computing an execution index before every method invocation, to be used as the ID of any thread spawned by that method. (The index remains unused if the method does not spawn any thread.) To handle the case that a method is invoked inside a loop, we also compute an execution index at the beginning of every basic block [37].

After an execution index (*i.e.*, ID) is computed, it is needed to be propagated to the thread spawned by the following method invocation. Moreover, once a thread gets an ID propagated to it, it needs to maintain the same ID for the duration of its lifetime. We leverage thread-local storage, `CallContext` in .NET [25], to achieve these functionalities.

`CallContext` can be considered as a key-value store unique for each *logical thread* of execution. It can store two types of data: *local* and *logical*. When a thread is created, the `CallContext` does not have any local data, but it inherits all logical data from its parents thread. Any local data, after its creation, remains in the thread's `CallContext` for the lifetime of the thread.

Listing 5 shows how FlakeRepro uses `CallContext` to compute a consistent thread ID. First, the method `UpdateExecIndex()`, which the Instrumenter injects (see Listing 3), computes an execution index and stores it as the current thread's `CallContext`'s logical data so that it can propagate to a spawned thread. Second, the method `GetThreadId()` returns the current thread's persistent ID as follows. On its first invocation on the current thread, it retrieves the ID from its logical data (which it inherits from parent thread) and stores the ID in its `CallContext`'s local data. Subsequent invocations of the method return the ID from the local data. Note that .NET also has the notion of `Task` [27] which is an operation that

```
public static void UpdateExecIndex(string method,
        int location) {
    string id =
        ComputeExecId(GetThreadId(),method,location);
    CallContext.LogicalSetData("ExecId", id);
}

public static string GetThreadId() {
    // If CallContext never has a value for the
    // key "ThreadId", this means GetThreadId()
    // is invoked first time.
    if (CallContext.GetData("ThreadId") == null) {
        string threaId
            = CallContext.LogicalGetData("ExecId");
        CallContext.SetData("ThreadId", threadId);
    }
    return CallContext.GetData("ThreadId");
}
```

Listing 5: Sample code to compute execution index by leveraging .NET `CallContext`

typically executes asynchronously on a thread pool. A consistent task ID can be computed similarly to thread ID and implemented with `AsyncLocal` [24].

Thread-local storage is supported by many other languages (*e.g.*, Java has `ThreadLocal` and `InheritableThreadLocal` [30, 31]) and the above idea can be implemented in those languages as well.

5 EVALUATION

5.1 Methodology

Flaky test benchmark. Our benchmark consists of a set of flaky unit tests from several .NET projects in Microsoft. Flaky tests were marked by CloudBuild, Microsoft's continuous integration system: when a test fails, CloudBuild re-executes it and if the second execution passes, the test is marked as flaky. We downloaded the test and application binaries along with the test failure messages, and used them in our experiment machine. Of all the downloads, we selected .NET tests that (1) could run in our experiment machine and passed at least once in 100 runs,[2] and (2) were multi-threaded (determined by collecting their execution trace and counting how many threads appear in the trace).

This process gave us a set of 31 flaky tests from 22 projects. For anonymity, we omit their real names and call them projects P1 to P22, and flaky tests FT1 to FT31. These flaky tests were our benchmark suite. The information of the projects' binary files and the tests are given in Table 1. The total number of instructions is shown to represent the size of the binaries. *Flaky degree* of a test is the fraction of times it failed in 1000 runs on our experiment machine. Some tests have a flaky degree of zero, implying that their errors are extremely hard to reproduce. FlakeRepro could still deterministically reproduce them, highlighting its value.

Experiment setting. We run FlakeRepro on a local machine with Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GHz, 12 Cores, 24 Logical Processors, 192 GB Memory, and 6 TB HDD with NTFS file system. The machine uses Microsoft Windows Server 2016 Datacenter as the operating system, and .NET 6.0 as the runtime.

[2]Despite our best efforts, some tests could not execute or executed with deterministic failures on our experiment machine due to missing dependencies or environment settings. For example, a missing dependency may be in CloudBuild's Global Assembly Cache (GAC) [23] and hence it is not included in the set of test binaries we downloaded. In a real use case, a developer's test machine should not have such issues.

Table 1: Projects built on CloudBuild and their flaky tests.

Project ID	#Binary Files	#Instructions	Test ID	Flaky Degree	Reproduced
P1	206	3,450,309	FT1	0	✓
			FT2	0	✓
			FT3	0	✓
			FT4	0	✓
P2	233	5,868,998	FT5	0	✓
P3	22	403,787	FT6	0	✓
P4	21	407,441	FT7	0	✓
P5	204	4,958,201	FT8	0	✓
P6	75	2,403,059	FT9	0	✓
P7	340	7,292,766	FT10	0	✓
P8	612	11,591,668	FT11	0	✓
P9	43	1,061,804	FT12	0.232	✓
P10	161	3,918,957	FT13	0	✓
			FT14	0	✓
			FT15	0	✓
P11	25	446,380	FT16	0	✓
			FT17	0	✓
P12	63	1,930,606	FT18	0.090	✓
			FT19	0.040	✓
P13	489	10,729,002	FT20	0	✓
P14	529	10,998,519	FT21	0	✓
P15	9	145,609	FT22	0	
P16	63	1,930,606	FT23	0	✓
P17	61	1,280,861	FT24	0.496	✓
P18	72	1,641,687	FT25	0.849	✓
P19	67	867,024	FT26	0.035	
P20	61	1,478,237	FT27	0.003	
P21	32	995,376	FT28	0	✓
P22	58	1,987,302	FT29	0	✓
			FT30	0	
			FT31	0	

5.2 Reproducing Result

Overall result. We used FlakeRepro to reproduce the errors of the test benchmark. From 31 flaky tests, FlakeRepro reproduced the errors of 26 tests under 6 minutes on average. The reproduced tests are shown in Table 1. Below, we discuss in details how FlakeAnalyzer and FlakeFinder perform on the tests that FlakeRepro can reproduce. For the tests that cannot be reproduced, we discuss them in Section 5.3.

FlakeAnalyzer. Table 2 shows how FlakeAnalyzer performs with the reproduced tests. On average, it took 50s (range: 5s-154s) per test and produced 138 (range: 3-873) critical accesses. As shown critical accesses often span over multiple assemblies, including test and product assemblies. In general, the number of critical accesses identified is relatively small compared to the number of instructions a test executes. This significantly expedites FlakeRepro's exploration since it limits its exploration to dynamic instances of the critical accesses only.

Table 2: The results of FlakeAnalyzer for the flaky tests that FlakeRepro can reproduce.

Test ID	Analysis Time (s)	#Critical Accesses	#Assemblies	Product	Test
FT1	15 s	87	4	✓	✓
FT2	68 s	39	3	✓	✓
FT3	91 s	15	2	✓	✓
FT4	74 s	15	2	✓	✓
FT5	42 s	43	3	✓	✓
FT6	61 s	19	2	✓	✓
FT7	11 s	4	1		✓
FT8	10 s	3	2	✓	✓
FT9	20 s	6	1		✓
FT10	15 s	5	1		✓
FT11	43 s	21	3	✓	✓
FT12	8 s	14	2	✓	✓
FT13	13 s	86	2	✓	✓
FT14	15 s	108	1		✓
FT15	14 s	108	1		✓
FT16	92 s	813	6	✓	✓
FT17	154 s	394	7	✓	✓
FT18	9 s	9	1		✓
FT19	84 s	80	4	✓	✓
FT20	17 s	56	5	✓	✓
FT21	40 s	175	5	✓	✓
FT23	96 s	873	8	✓	✓
FT24	70 s	163	5	✓	✓
FT25	5 s	18	1		✓
FT28	106 s	342	4	✓	✓
FT29	135 s	84	5	✓	✓

Table 3: The result of FlakeFinder for the flaky tests that FlakeRepro can reproduce.

Test ID	#Threads	#Access Instances	Exploration Time (s)	#Trials
FT1	3	206	34 s	1
FT2	6	46	235 s	7
FT3	6	22	22 s	1
FT4	5	23	20 s	1
FT5	9	200	59 s	1
FT6	198	760	324 s	26
FT7	2	5	9 s	1
FT8	10	12	20 s	1
FT9	2	7	23 s	2
FT10	2	4	81 s	1
FT11	8	40,307	1,153 s	14
FT12	2	43	518 s	3
FT13	3	108	129 s	5
FT14	360	38,091	537 s	4
FT15	14	776	398 s	11
FT16	2	19	61 s	2
FT17	509	147,679	246 s	3
FT18	5	82	32 s	1
FT19	4	1,017	334 s	13
FT20	9	173	117 s	1
FT21	4	18,763	2,454 s	32
FT23	2	2976	37 s	1
FT24	7	107,664	1,393 s	25
FT25	3	18	157 s	4
FT28	33	884	66 s	1
FT29	7	276	61 s	7

FlakeFinder. Table 3 shows how FlakeFinder performs on the reproduced tests. It shows how many threads each test creates and how many dynamic instances of the critical accesses it makes. Roughly half the tests involve 5 threads and > 100 access instances, highlighting the complexity of the tests. Yet, on average, FlakeFinder can reproduce the errors in ≈ 6 minutes (range: 9s-40m), with < 7 trials on average (range: 1-32). As aforementioned in Section 3.2, each trial executes a test with one partial ordering of a pair of critical accesses, and thus the number of trials represents an over-approximation of the number of interleavings explored. This number is significantly smaller than the number of interleavings explored by general concurrency bug finding tools (*e.g.*, CHESS explores thousands of interleavings).

Interestingly, for multiple tests, Explorer can reproduce the failure by only few or even only one trial. These tests fail because their assertions execute before other untimely memory accesses, and FlakeFinder prioritizes this pattern. In addition, there are more than one interleavings that yield the reported error so FlakeFinder quickly finds any one of them.

5.3 Discussion

For the tests that FlakeRepro could not reproduce, we saw two outcomes. Firstly, FlakeFinder explores all interleavings and cannot reproduce the errors, which is the case of FT22, FT27, and FT30. This can happen because (1) FlakeAnalyzer's static and dynamic analysis may fail to determine *all* critical memory accesses and miss the one that causes the test failure. Moreover, (2) FlakeFinder's heuristic may prune out the buggy interleaving. This can happen because Explorer's happens-before inference is inaccurate, or the failure requires more than two critical access instances to execute in a certain order (FlakeFinder controls only a pair of instances for each trial).

For the second outcome, FlakeFinder could not finish exploring in the timeout limit, which we set to 3 hours. This happens with the test FT26, and FT31. This outcome indicates that more advanced pruning and prioritizing strategies are needed in order to reproduce these failures.

5.4 Flaky Test Example

Listing 1 shows an example flaky test that FlakeRepro could reproduce. We here present another example, as shown in Listing 6. The test checks the eviction policy of `Cache`. The test first fills `cache` until it is nearly full (line 5). It then continues to add more items to kick out a background eviction thread (`AddUntilEviction()` at line 10). The test finally checks if all the oldest keys are evicted (line 12). The test does not use any synchronization between the completion of the eviction thread and the assertions in the main thread: if the assertion happens before the eviction thread completes evicting the oldest items, the test fails. FlakeRepro reports concurrency

```
1  [TestMethod]
2  public void TestCacheEviction() {
3      Cache cache;
4      // Fill cache until it is nearly full.
5      FillCache(cache);
6      List oldestKeys = cache.GetOldestKeys();
7      // Add more items until it exceeds cache capacity.
8      // This triggers a background eviction thread so
9      // all items in oldestKeys are evicted.
10     AddUntilEviction(cache);
11     foreach (key in oldestKeys)
12         Assert.IsFalse(cache.Contains(key));
13 }
14
15 private void AddUntilEviction(Cache cache,
16     bool waitEviction = false);
```

Listing 6: Flaky test example that is caused by a missing synchronization

between the two threads, identifies their buggy interleaving, and deterministically reproduces the failure.

This flakiness can be fixed by waiting until the eviction thread finishes before starting the assertions. `AddUntilEviction()` method already has an optional second parameter that, if `true`, makes the method wait until the background eviction thread finishes before return. However, the developer forgot to pass the second argument, which caused the flakiness. The fix is simply to pass `true` as the second argument.

6 FLAKEREPRO IN MICROSOFT

FlakeRepro has been piloted in Microsoft as an offline tool. After CloudBuild reports a flaky test, a developer downloads the test binaries and applies FlakeRepro to deterministically reproduce the test failure. If the test is not multi-threaded, or if FlakeRepro's exploration algorithm cannot reproduce the target error message, FlakeRepro outputs its inability to reproduce the flakiness. Otherwise, it produces instrumented test binaries, a buggy interleaving, and additional information such as critical shared memory accesses and stack traces of threads while accessing the shared memory. The developer can investigate the given ordering and the stack traces, and runs the test from the instrumented binaries with a debugger attached to understand the failure in detail. She then removes the flakiness, and if wanted, she can run FlakeRepro again after the fix to verify that the test is rid of the flakiness.

Our next step is to enable FlakeRepro in CloudBuild, Microsoft's build pipeline. When a test is found flaky, the pipeline will run FlakeRepro, and attach the output it generates with the bug report so the responsible developer can have a quick detail to start investigating. And to facilitate developers to use FlakeRepro offline, we are packing FlakeRepro as a .NET tool [26] for developers to easily integrate FlakeRepro in their .NET development environment.

7 RELATED WORK

Flaky test study. As aforementioned, there are several studies and reports on flaky tests in both open source and industry [3, 9, 11, 12, 20, 34]. These studies has a variety of implications for current and future works. They show how critical flaky tests are, and guide us how to combat existing flaky tests and prevent new ones added to test suites. Due to the pervasiveness of concurrency-related flaky tests shown in the previous studies, including in Microsoft, we are motivated to prioritize this category of flaky tests, and FlakeRepro is introduced to fight them.

Concurrency bug detection. Concurrency bugs are common in modern software [6, 18]. This type of bugs are hard to detect during development process. One approach is a model checking [14, 19, 28, 39]. Model checker explores possible interleaving to see if any of them could fail the systems. This approach suffers from the *state space explosion*. The recent works are focusing on prioritizing and pruning the state space [14, 19]. Fuzzing is another approach to detect concurrency bugs [17, 32, 35]. It *fuzzes* the software by injecting random delay to discover new interleavings. Because fuzzing is done randomly, it wastes computing resources. TVSD [17] is a recent fuzzing tool that can infer happens-before relationship, and smartly injects delays to improve efficiency. Program analysis is also used to detect concurrency bugs. Several works are based on both static and dynamic analysis [40, 41].

Failure reproducing. Failure reproducing is the first significant step to fix bugs [42]. A bug report usually comes with only log files, and a developer needs to construct workload to exercise buggy code herself. Pensieve helps construct the workload automatically [42], however, the developer still needs to manually enforce non-determinism, if any. Another work is record and replay which captures non-determinism during execution [2, 38]. This solution records a failed execution trace to replay the failure later for debugging, however, execution trace recording is costly and cannot be always enabled in production. REPT leverages hardware tracing to mitigate this problem [2].

Flaky test detection and root causing. Several prior attempts introduce techniques for detecting flaky tests [1, 10]. DeFlaker [1] looks at the recent changes and determines if the failed tests are affected by the recent code changes to identify flaky tests, this reduces resources needed to determine flakiness. iDFlakies [10], detects flaky tests due to test order dependency, which partially help root cause the failures. RootFinder [8] is a tool that receives execution traces of both passed and failed run, and computes the executing conditions that make the tests fail. A developer can use the hints from RootFinder to reproduce the failures, but unlike FlakeRepro, she needs to enforce all the conditions herself.

8 CONCLUSION

In this paper, we have presented FlakeRepro that helps developers reproduce a failed execution of a flaky test caused by concurrency. FlakeRepro combines static and dynamic analysis to quickly identify an interleaving that fails a flaky test with the same original error message. FlakeRepro is efficient: it can reproduce a failed execution after exploring few tens of interleavings. FlakeRepro integrates well with existing systems: it automatically instruments test binaries that can run on existing and unmodified test pipelines. We have implemented FlakeRepro for .NET. In an experiment with 22 Microsoft projects, FlakeRepro could reproduce 26 of total 31 concurrent-related flaky tests, after exploring only < 7 interleavings and within 6 minutes on average.

REFERENCES

[1] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests. In *Proceedings of the IEEE/ACM 40th International Conference on Software Engineering (ICSE '18)*. https://doi.org/10.1145/3180155.3180164

[2] Weidong Cui, Xinyang Ge, Baris Kasikci, Ben Niu, Upamanyu Sharma, Ruoyu Wang, and Insu Yun. 2018. REPT: Reverse Debugging of Failures in Deployed Software. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*.

[3] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding Flaky Tests: The Developer's Perspective. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. New York, NY, USA, 830–840. https://doi.org/10.1145/3338906.3338945

[4] Hamed Esfahani, Jonas Fietz, Qi Ke, Alexei Kolomiets, Erica Lan, Erik Mavrinac, Wolfram Schulte, Newton Sanches, and Srikanth Kandula. 2016. CloudBuild: Microsoft's Distributed and Caching Build Service. In *Proceedings Of the 38th ACM International Conference on Software Engineering (ICSE '16)*. Austin, TX, USA. https://doi.org/10.1145/2889160.2889222

[5] Jb Evain. 2022. Mono.Cecil. https://github.com/jbevain/cecil/.

[6] Haryadi S. Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, Thanh Do, Jeffry Adityatama, Kurnia J. Eliazar, Agung Laksono, Jeffrey F. Lukman, Vincentius Martin, and Anang D. Satria. 2014. What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems. In *Proceedings of the 5th ACM Symposium on Cloud Computing (SoCC '14)*. Seattle, WA, USA. https://doi.org/10.1145/2670979.2670986

[7] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. Memon. 2020. Modeling and Ranking Flaky Tests at Apple. In *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE '20)*. Los Alamitos, CA, USA, 110–119. https://doi.org/10.1145/3377813.3381370

[8] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root Causing Flaky Tests in a Large-Scale Industrial Setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '19)*. Beijing, China. https://doi.org/10.1145/3293882.3330570

[9] Wing Lam, Kivanc Muslu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A Study on the Lifecycle of Flaky Tests. In *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE '20)*. https://doi.org/10.1145/3377811.3381749

[10] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *Proceedings of the 12th IEEE Conference on Software Testing, Validation and Verification (ICST '19)*. https://doi.org/10.1109/ICST.2019.00038

[11] Wing Lam, Stefan Winter, Angello Astorga, Victoria Stodden, and Darko Marinov. 2020. Understanding Reproducibility and Characteristics of Flaky Tests Through Test Reruns in Java Projects. In *Proceedings of 31st IEEE International Symposium on Software Reliability Engineering (ISSRE '20)*. https://doi.org/10.1109/ISSRE5003.2020.00045

[12] Wing Lam, Stefan Winter, Anjiang Wei, Tao Xie, Darko Marinov, and Jonathan Bell. 2020. A Large-Scale Longitudinal Study of Flaky Tests. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 202 (nov 2020). https://doi.org/10.1145/3428270

[13] Leslie Lamport. 1978. Time, Clocks and the Ordering of Events in a Distributed System. *Communications of the ACM 21, 7 (July 1978), 558-565. Reprinted in several collections, including Distributed Computing: Concepts and Implementations, McEntire et al., ed. IEEE Press, 1984.* (1978), 558–565. https://doi.org/10.1145/359545.359563

[14] Tanakorn Leesatapornwongsa, Mingzhe Hao, Pallavi Joshi, Jeffrey F. Lukman, and Haryadi S. Gunawi. 2014. SAMC: Semantic-Aware Model Checking for Fast Discovery of Deep Bugs in Cloud Systems. In *Proceedings of the 11th Symposium on Operating Systems Design and Implementation (OSDI '14)*. Broomfield, CO, USA.

[15] Tanakorn Leesatapornwongsa, Jeffrey F. Lukman, Shan Lu, and Haryadi S. Gunawi. 2016. TaxDC: A Taxonomy of Non-Deterministic Concurrency Bugs in Datacenter Distributed Systems. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '16)*. https://doi.org/10.1145/2954679.2872374

[16] Guangpu Li, Dongjie Chen, Shan Lu, Madanlal Musuvathi, and Suman Nath. 2021. SherLock: unsupervised synchronization-operation inference. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. 314–328. https://doi.org/10.1145/3445814.3446754

[17] Guangpu Li, Shan Lu, Madanlal Musuvathi, Suman Nath, and Rohan Padhye. 2019. Efficient scalable thread-safety-violation detection: finding thousands of concurrency bugs during testing. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 162–180. https://doi.org/10.1145/3341301.3359638

[18] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. 2008. Learning from Mistakes — A Comprehensive Study on Real World Concurrency Bug Characteristics. In *Proceedings of the 13th international conference on Architectural support for programming languages and operating systems (ASPLOS '08)*. Seattle, WA, USA. https://doi.org/10.1145/1353535.1346323

[19] Jeffrey F. Lukman, Huan Ke, Cesar A. Stuardo, Riza O. Suminto, Daniar H. Kurniawan, Dikaimin Simon, Satria Priambada, Chen Tian, Feng Ye, Tanakorn Leesatapornwongsa, Aarti Gupta, Shan Lu, and Haryadi S. Gunawi. 2019. FlyMC: Highly Scalable Testing of Complex Interleavings in Distributed Systems. In *Proceedings of the 14th EuroSys Conference 2019 (EuroSys '19)*. Dresden, Germany. https://doi.org/10.1145/3302424.3303986

[20] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '14)*. New York, NY, USA, 643–653. https://doi.org/10.1145/2635868.2635920

[21] Friedemann Mattern. 1989. Virtual time and global states of distributed systems. (1989).

[22] John Micco. 2017. The State of Continuous Integration Testing at Google. https://www.aster.or.jp/conference/icst2017/program/jmicco-keynote.pdf.

[23] Microsoft. 2021. Global Assembly Cache. https://docs.microsoft.com/en-us/dotnet/framework/app-domains/gac.

[24] Microsoft. 2022. AsyncLocal<T> Class (System.Threading). https://docs.microsoft.com/en-us/dotnet/api/system.threading.asynclocal-1.

[25] Microsoft. 2022. CallContext Class (System.Runtime.Remoting.Messaging). https://docs.microsoft.com/en-us/dotnet/api/system.runtime.remoting.messaging.callcontext.

[26] Microsoft. 2022. How to manage .NET tools. https://docs.microsoft.com/en-us/dotnet/core/tools/global-tools.

[27] Microsoft. 2022. Task Class (System.Threading.Tasks). https://docs.microsoft.com/en-us/dotnet/api/system.threading.tasks.task.

[28] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. 2008. Finding and Reproducing Heisenbugs in Concurrent Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI '08)*. San Diego, CA, USA, 267–280.

[29] Armin Najafi, Peter C. Rigby, and Weiyi Shang. 2019. Bisecting Commits and Modeling Commit Risk during Testing. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. New York, NY, USA, 279–289. https://doi.org/10.1145/3338906.3338944

[30] Oracle. 2018. InheritableThreadLocal. https://docs.oracle.com/javase/10/docs/api/java/lang/InheritableThreadLocal.html.

[31] Oracle. 2018. ThreadLocal. https://docs.oracle.com/javase/10/docs/api/java/lang/ThreadLocal.html.

[32] Soyeon Park, Shan Lu, and Yuanyuan Zhou. 2009. CTrigger: Exposing Atomicity Violation Bugs from Their Hiding Places. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*. Washington DC, USA, 25–36. https://doi.org/10.1145/2528521.1508249

[33] Md Tajmilur Rahman and Peter C. Rigby. 2018. The Impact of Failing, Flaky, and High Failure Tests on the Number of Crash Reports Associated with Firefox Builds. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)*. 857–862. https://doi.org/10.1145/3236024.3275529

[34] Maaz Hafeez Ur Rehman and Peter C. Rigby. 2021. Quantifying No-Fault-Found Test Failures to Prioritize Inspection of Flaky Tests at Ericsson. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. Athens, Greece, 1371–1380. https://doi.org/10.1145/3468264.3473930

[35] Koushik Sen. 2008. Race Directed Random Testing of Concurrent Programs. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '08)*. 11–21. https://doi.org/10.1145/1375581.1375584

[36] Mark Weiser. 1981. Program Slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE '81)*. 439–449. https://doi.org/10.1109/TSE.1984.5010248

[37] Bin Xin, William N. Sumner, and Xiangyu Zhang. 2008. Efficient Program Execution Indexing. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '08)*. Tucson, AZ, USA, 238–248. https://doi.org/10.1145/1379022.1375611

[38] Maysam Yabandeh, Nikola Knezevic, Dejan Kostic, and Viktor Kuncak. 2010. CrystalBall: Predicting and Preventing Inconsistencies in Deployed Distributed Systems. In *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation (NSDI' 10)*. https://doi.org/10.1145/1731060.1731062

[39] Junfeng Yang, Tisheng Chen, Ming Wu, Zhilei Xu, Xuezheng Liu, Haoxiang Lin, Mao Yang, Fan Long, Lintao Zhang, and Lidong Zhou. 2009. MODIST: Transparent Model Checking of Unmodified Distributed Systems. In *Proceedings of the 6th Symposium on Networked Systems Design and Implementation (NSDI '09)*. 213–228.

[40] Wei Zhang, Junghee Lim, Ramya Olichandran, Joel Scherpelz, Guoliang Jin, Shan Lu, and Thomas Reps. 2011. ConSeq: Detecting Concurrency Bugs through

Sequential Errors. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '11)*. 251–264. https://doi.org/10.1145/1961295.1950395

[41] Wei Zhang, Chong Sun, and Shan Lu. 2010. ConMem: Detecting Severe Concurrency Bugs through an Effect-Oriented Approach. In *Proceedings of the Fifteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '10)*. 179–192. https://doi.org/10.1145/1735971.1736041

[42] Yongle Zhang, Serguei Makarov, Xiang Ren, David Lion, and Ding Yuan. 2017. Pensieve: Non-Intrusive Failure Reproduction for Distributed Systems Using the Event Chaining Approach. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. Shanghai, China, 19–33. https://doi.org/10.1145/3132747.3132768

[43] Edgardo Zoppi. 2017. Analysis.NET. https://github.com/edgardozoppi/analysis-net.